

Name: Avinash Rajput

Task Assigned by : Mithun

Task 1: Create a Sample Maven Project using CLI commands and build the code using Maven Goals. Copy the code to the Linux Server (Install Java and Maven) and build the code.

Step 1: Create a Sample Maven Project on Your Local Machine (Using CLI)

```
mvn archetype:generate -DgroupId=com.example.demo \
-DartifactId=sample-maven-project \
-DarchetypeArtifactId=maven-archetype-quickstart \
-DinteractiveMode=false
```

This will generate a basic Java app inside a folder named sample-maven-project.

```
maven-quickstart-1.0.jar (4.3 kB at 100 kB/s)
[INFO] -----
[INFO] Using following parameters for creating project from old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO] -----
[INFO] Parameter: basedir, Value: C:\Users\vivek\maven-proj
[INFO] Parameter: package, Value: com.example.demo
[INFO] Parameter: groupId, Value: com.example.demo
[INFO] Parameter: artifactId, Value: sample-maven-project
[INFO] Parameter: packageName, Value: com.example.demo
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from old (1.x) Archetype in dir: C:\Users\vivek\maven-proj\sample-maven-project
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 12.341 s
[INFO] Finished at: 2025-07-16T10:02:08+05:30
[INFO] -----
avinash@LAPTOP-420TVVSA ~/maven-proj $ |
```

Step 2: Build the Project Locally

```
cd sample-maven-project
```

```
mvn clean package
```

```
avinash@LAPTOP-420TVVSA ~/maven-proj $ ls
sample-maven-project/
avinash@LAPTOP-420TVVSA ~/maven-proj $ cd sample-maven-project/
avinash@LAPTOP-420TVVSA ~/maven-proj/sample-maven-project $ ls
pom.xml src/
avinash@LAPTOP-420TVVSA ~/maven-proj/sample-maven-project $ mvn clean package
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::staticFieldBase has been called by com.google.inject in
avinash@LAPTOP-420TVVSA ~/maven-proj/sample-maven-project $ ls
pom.xml src/ target/
avinash@LAPTOP-420TVVSA ~/maven-proj/sample-maven-project $
```

Step 3: Copy Project to EC2 Server (Linux)

```
scp -i demo-key.pem -r sample-maven-project ubuntu@13.232.47.93:/home/ubuntu/
```

```

avinash@LAPTOP-420TVVSA ~/maven-proj $ scp -i demo-key.pem -r sample-maven-project ubuntu@13.232.47.93:/home/ubuntu/
The authenticity of host '13.232.47.93 (13.232.47.93)' can't be established.
ED25519 key fingerprint is SHA256:28m+v2u3fsv1m8siepTD8IK+JZ1dRQFXVWOP6F3e5A.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '13.232.47.93' (ED25519) to the list of known hosts.
pom.xml          100% 687      7.4kB/s  00:00
app.java         100% 192      8.2kB/s  00:00
AppTest.java    100% 682      6.5kB/s  00:00
app.class        100% 547     23.5kB/s  00:00
pom.properties  100% 81       0.3kB/s  00:00
createdFiles.lst 100% 27       0.3kB/s  00:00
inputFiles.lst   100% 88       0.8kB/s  00:00
createdFiles.lst 100% 31       0.3kB/s  00:00
inputFiles.lst   100% 92       0.9kB/s  00:00
sample-maven-project-1.0-SNAPSHOT.jar 100% 2304    21.5kB/s  00:00
com.example.demo.AppTest.txt 100% 302      2.9kB/s  00:00
TEST-com.example.demo.AppTest.xml 100% 6576    75.2kB/s  00:00
AppTest.class    100% 611     10.1kB/s  00:00

```

Step 4: Connect to EC2 Server

```

avinash@LAPTOP-420TVVSA ~/maven-proj $ ssh -i demo-key.pem ubuntu@13.232.47.93
welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.8.0-1029-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed Jul 16 05:27:27 UTC 2025

System load:  0.08      Processes:           107
Usage of /:   25.6% of 6.71GB   Users logged in:    1
Memory usage: 20%        IPv4 address for enx0: 10.0.1.16
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.
0 updates can be applied immediately.

```

Step 5: Install Java & Maven on Ubuntu Server

sudo apt update

sudo apt install openjdk-17 -y

java -version

sudo apt install maven -y

mvn -version

```

ubuntu@ip-10-0-1-16:~$ mvn -v
Apache Maven 3.8.7
Maven home: /usr/share/maven
Java version: 21.0.7, vendor: Ubuntu, runtime: /usr/lib/jvm/java-21-openjdk-amd64
Default locale: en, platform encoding: UTF-8
OS name: "linux", version: "6.8.0-1029-aws", arch: "amd64", family: "unix"
ubuntu@ip-10-0-1-16:~$ java -version
openjdk version "21.0.7" 2025-04-15
OpenJDK Runtime Environment (build 21.0.7+6-Ubuntu-0ubuntu124.04)
OpenJDK 64-Bit Server VM (build 21.0.7+6-Ubuntu-0ubuntu124.04, mixed mode, sharing)
ubuntu@ip-10-0-1-16:~$

```

Step 6: Build Maven Code on EC2 Server follow same process as local

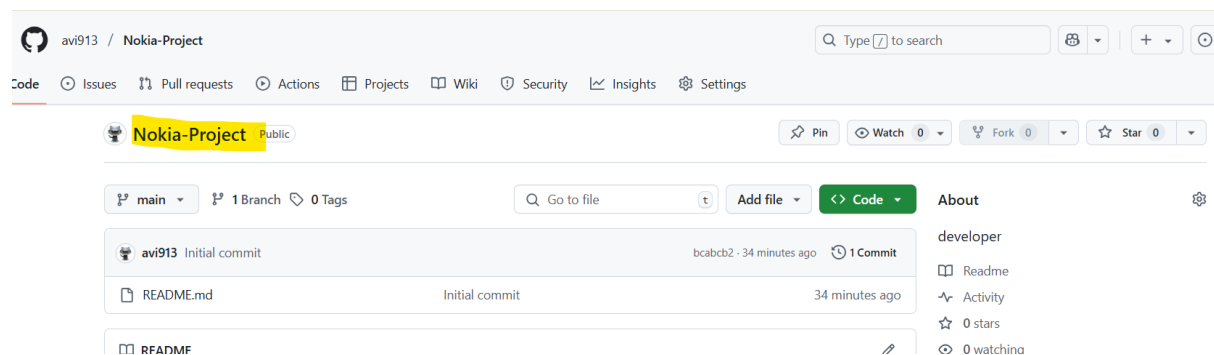
```

Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plex
6.4 MB/s)
[INFO] Installing /home/ubuntu/sample-maven-project/target/sample-maven-project
mple/demo/sample-maven-project/1.0-SNAPSHOT/sample-maven-project-1.0-SNAPSHOT.j
[INFO] Installing /home/ubuntu/sample-maven-project/pom.xml to /home/ubuntu/.m2
APSHOT/sample-maven-project-1.0-SNAPSHOT.pom
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 7.242 s
[INFO] Finished at: 2025-07-16T05:52:34Z
[INFO] -----
ubuntu@ip-10-0-1-16:~/sample-maven-project$

```

Task 2: Create a GIT Repository named Nokia-Project with branches (Test, development, pre-uat, uat, pre-prod, production, feature and hotfix) in the github repository. Authenticate to github by generating SSH credentials. Clone the repo and perform all git tasks.

Create a GitHub Repository -name:Nokia-Project

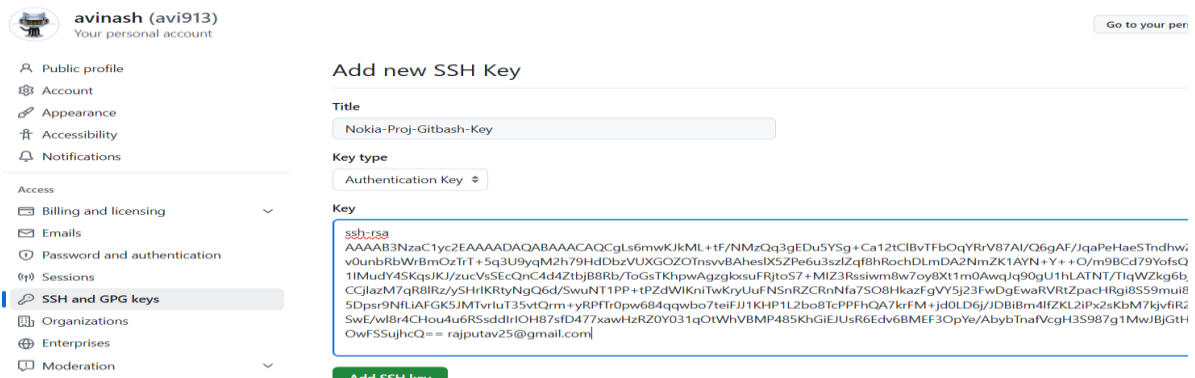


Generate ssh key :[ssh-keygen -t rsa -b 4096 -C "rajputav25@gmail.com"]

View the key: [cat /c/Users/vivek/.ssh/id_rsa.pub]

```
avinash@LAPTOP-420TVVSA ~ $ ssh-keygen -t rsa -b 4096 -C "rajputav25@gmail.com"
Generating public/private rsa key pair.
Enter file in which to save the key (/c/Users/vivek/.ssh/id_rsa):
/c/Users/vivek/.ssh/id_rsa already exists.
Overwrite (y/n)? y
Enter passphrase for "/c/Users/vivek/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /c/Users/vivek/.ssh/id_rsa
Your public key has been saved in /c/Users/vivek/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:B0cQ0JuXnD3P8JBeeQ2nLB02wzXH8MDpyFT07oFSo48 rajputav25@gmail.com
The key's randomart image is:
+---[RSA 4096]---+
|.ooo. o++|
|. . +o*o+|
|.+.0o0oBo|
|oo=o#* o|
|S..%o.|
|. +.+|
|. E .|
+---[SHA256]---+
avinash@LAPTOP-420TVVSA ~ $ cat /c/Users/vivek/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQACgLS6mwKJkML+tF/NMzQq3gEDu5YSg+Ca12tC1BvTFb0qYRrV87AI/Q6gAF/JqaPeHaeSTndhw2f0F6
+O/m9BCd79YofSjwL7Ervu1IMudY4SKqsJKJ/zucVsSEcQc4d4ZtbjB8Rb/ToGsTKhpwAgzgkxsuFRjtoS7+MIZ3Rssiwm8w/oy8Xt1m0AwqJq90gU
7S08HkazFgVY5j23FwDgEwaRVrtZpacHRgi8S59mu18i9dg0EGt5Dpsr9NFLiAFGK5JMTvr1uT35vtQrm+yRPfTr0pw684qqwbo7teifJJ1KHP1L2bo8T
awHzRZOY031q0tWhVBMP485KhGiEJUsR6Edv6BMEF30pYe/AbybTnafVcgH3S987g1MwJBjGtH129KPQ5jOwFSSujhcQ== rajputav25@gmail.com
avinash@LAPTOP-420TVVSA ~ $
```

Add SSH Key to GitHub Settings



Clone the Repo Using SSH

[git clone [git@github.com:avi913/Nokia-Project.git](https://github.com/avi913/Nokia-Project.git)]

```
Interalized Empty Git Repository https://github.com/avi913/Nokia-Project.git
avinash@LAPTOP-420TVVSA ~/nokia $ git clone git@github.com:avi913/Nokia-Project.git
Cloning into 'Nokia-Project'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
avinash@LAPTOP-420TVVSA ~/nokia $
```

Create All Required Branches

git checkout -b Test

git push -u origin Test

git checkout main

git checkout -b development

git push -u origin development

git checkout main

git checkout -b pre-uat

git push -u origin pre-uat

git checkout main

git checkout -b uat

git push -u origin uat

git checkout main

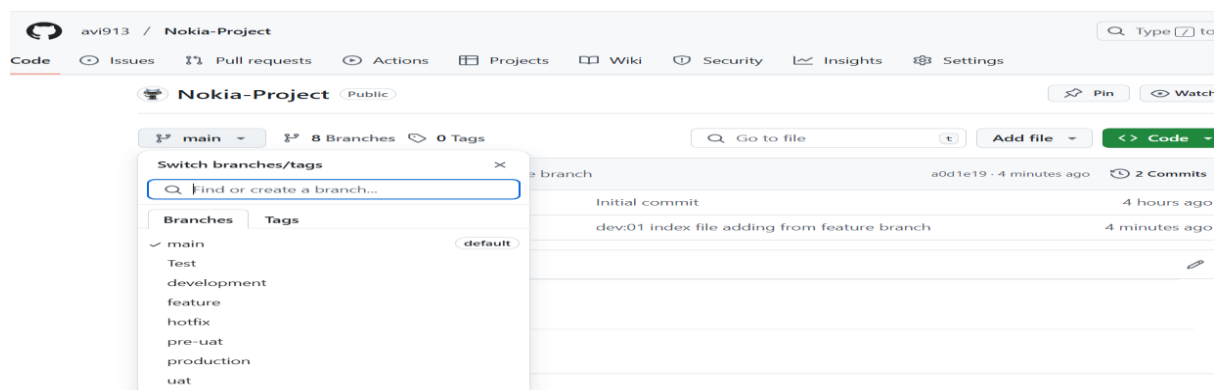
git checkout -b pre-prod

git push -u origin pre-prod

git checkout main

git checkout -b production

git push -u origin production



Task 3: Create a new repo called merge-conflict and clone it to 2 different ec2 instances and then start working on the same piece of code from both the instances and then try to push the changes to GitHub.

Created merge-conflict branch

```
avinash@LAPTOP-420TVVSA ~/nokia/Nokia-Project $ git push origin merge-conflict
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'merge-conflict' on GitHub by visiting:
remote:   https://github.com/avi913/Nokia-Project/pull/new/merge-conflict
remote:
To github.com:avi913/Nokia-Project.git
 * [new branch]      merge-conflict -> merge-conflict
avinash@LAPTOP-420TVVSA ~/nokia/Nokia-Project $ |
```

Added the 3rd line in 1 ec2

```
avinash@LAPTOP-420TVVSA ~/nokia/Nokia-Project $ cat index.txt
hello from feature branch
add the file
good morning
avinash@LAPTOP-420TVVSA ~/nokia/Nokia-Project $ git branch
  development
  feature
  hotfix
  main
  merge-conflict
  pre-uat
  production
  uat
avinash@LAPTOP-420TVVSA ~/nokia/Nokia-Project $

no changes added to commit (use "git add" and/or "git commit -a")
avinash@LAPTOP-420TVVSA ~/nokia/Nokia-Project $ git add .
avinash@LAPTOP-420TVVSA ~/nokia/Nokia-Project $ git commit -m 'dev:02 file 3rd line added'
git: 'commit' is not a git command. See 'git --help'.

The most similar command is
  commit
avinash@LAPTOP-420TVVSA ~/nokia/Nokia-Project $ git commit -m 'dev:02 file 3rd line added'
[merge-conflict 66521bd] dev:02 file 3rd line added
1 file changed, 2 insertions(+)
avinash@LAPTOP-420TVVSA ~/nokia/Nokia-Project $ git push origin merge-conflict
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 331 bytes | 331.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:avi913/Nokia-Project.git
   bcb2cb2..66521bd merge-conflict -> merge-conflict
avinash@LAPTOP-420TVVSA ~/nokia/Nokia-Project $ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
avinash@LAPTOP-420TVVSA ~/nokia/Nokia-Project $ git merge merge-conflict
Updating a0d1e19..66521bd
Fast-forward
   index.txt | 2 ++
   1 file changed, 2 insertions(+)
avinash@LAPTOP-420TVVSA ~/nokia/Nokia-Project $ |
```

Added the content from dev branch in same file with the same line now it will show merge conflict

```

a0d1e19..0360405 development -> development
avinash@LAPTOP-420TVVSA ~/nokia/Nokia-Project $ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
avinash@LAPTOP-420TVVSA ~/nokia/Nokia-Project $ git merge development
Auto-merging index.txt
CONFLICT (content): Merge conflict in index.txt
Automatic merge failed; fix conflicts and then commit the result.
avinash@LAPTOP-420TVVSA ~/nokia/Nokia-Project $

avinash@LAPTOP-420TVVSA ~/nokia/Nokia-Project $ cat index.txt
hello from feature branch
<<<<<< HEAD
add the file
good morning
=====
add the file
good morning
>>>>>> development
avinash@LAPTOP-420TVVSA ~/nokia/Nokia-Project $ |
```

We need to resolve the merge conflict and then push to main

```
avinash@LAPTOP-420TVVSA ~/nokia/Nokia-Project $ git commit -m 'dev:03 modified file from main'
[main 2c5d90b] dev:03 modified file from main
avinash@LAPTOP-420TVVSA ~/nokia/Nokia-Project $ git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 367 bytes | 367.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:avi913/Nokia-Project.git
   66521bd..2c5d90b  main -> main
avinash@LAPTOP-420TVVSA ~/nokia/Nokia-Project $ |
```

Task 4: Install [Jenkins-Installation-Script](#) on aws ec2 and then create two freestyle jobs Project 1 & Project 2 such that Project 2 gets triggered automatically once the Project 1 build is complete. 4.1 - Configure swap memory to avoid memory shrinkage

Step 1: Launch EC2 Instance

1. Open AWS Console → EC2 → Launch Instance
2. Configuration: Security Group: Allow these ports: SSH: 22, HTTP: 80, Custom TCP: 8080 (for Jenkins)

Step 2: SSH Into EC2 & Update System

```
avinash@LAPTOP-420TVVSA ~/Downloads $ ssh -i "demo-key.pem" ubuntu@ec2-65-0-71-81.ap-south-1.compute.amazonaws.com
ssh: connect to host ec2-65-0-71-81.ap-south-1.compute.amazonaws.com port 22: Connection timed out
avinash@LAPTOP-420TVVSA ~/Downloads $ ssh -i "demo-key.pem" ubuntu@13.126.21.130
The authenticity of host '13.126.21.130 (13.126.21.130)' can't be established.
ED25519 key fingerprint is SHA256:D8jaxyj+lJxNsJlh1T7ePq7nXefhknYAc3d1Wws5660.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '13.126.21.130' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.8.0-1029-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Jun 10 10:02:26 UTC 2025
```

Cmd: sudo apt update && sudo apt upgrade -y

Step 4: Install Jenkins

```
wget -q -O - https://pkg.jenkins.io/debian/jenkins.io.key | sudo tee \
/usr/share/keyrings/jenkins-keyring.asc > /dev/null && \
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
https://pkg.jenkins.io/debian binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null && \
sudo apt update && sudo apt install -y openjdk-17-jdk Jenkins
```

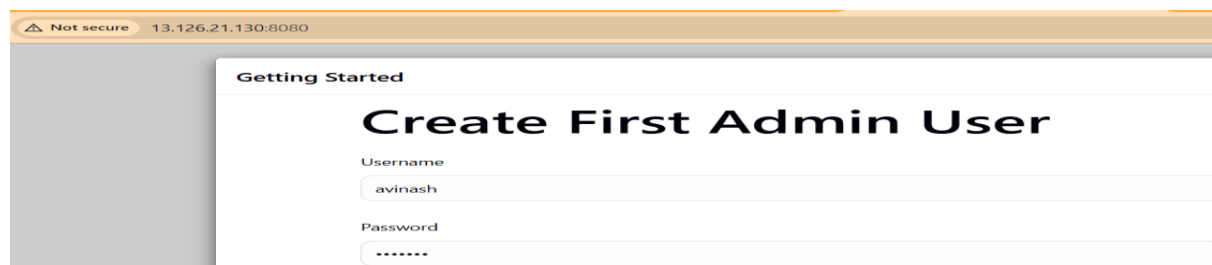
sudo systemctl start Jenkins

sudo systemctl enable Jenkins

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-10-0-1-4:~$ sudo cat /var/lib/jenkins/secrets/initialAdminPassword
cat: /var/lib/jenkins/secrets/initialAdminPassword: No such file or directory
ubuntu@ip-10-0-1-4:~$ cat /var/lib/jenkins/secrets/initialAdminPassword
cat: /var/lib/jenkins/secrets/initialAdminPassword: Permission denied
ubuntu@ip-10-0-1-4:~$ sudo cat /var/lib/jenkins/secrets/initialAdminPassword
8447bc12dfFc446baaabd013555ab01a
ubuntu@ip-10-0-1-4:~$ ^C
ubuntu@ip-10-0-1-4:~$ sudo systemctl status jenkins
jenkins.service - Jenkins Continuous Integration Server
Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
Active: active (running) since Fri 2025-07-18 05:20:58 UTC; 2min 47s ago
Main PID: 3446 (java)
Tasks: 42 (limit: 1124)
Memory: 438.6M (peak: 446.0M swap: 1.2M swap peak: 1.2M)
CPU: 29.984s
CGroup: /system.slice/jenkins.service
└─3446 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --web
Jul 18 05:23:41 ip-10-0-1-4 jenkins[3446]: 2025-07-18 05:23:41.131+0000 [id=82] INFO
Jul 18 05:23:41 ip-10-0-1-4 jenkins[3446]: 2025-07-18 05:23:41.300+0000 [id=82] INFO
Jul 18 05:23:43 ip-10-0-1-4 jenkins[3446]: 2025-07-18 05:23:43.251+0000 [id=82] INFO
```

Step 5: Access Jenkins Web UI

1. Go to [http:// 13.126.21.130:8080](http://13.126.21.130:8080)



Step 6: Create Jenkins Jobs

- Job 1: Project-1:Dashboard → New Item → Project-1,Type: Freestyle project

In Build section:add the simple script

Bash cmd: echo "This is Project 1"

sleep 5

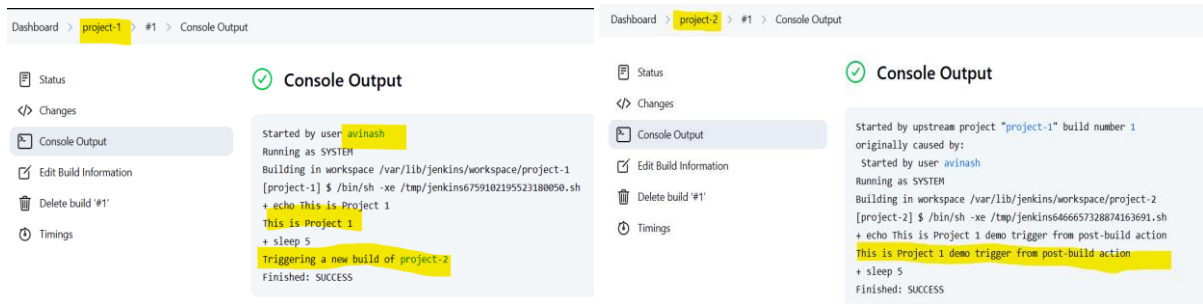
S	W	Name ↓	Last Success	Last Failure	Last Duration	
...	☀	project-1 ▾	N/A	N/A	N/A	▶
...	☀	project-2	N/A	N/A	N/A	▶

Step 7: Trigger Project-2 After Project-1

1. Go to Project-1 → Configure,Scroll to Post-build Actions,Click “Add post-build action” → “Build other projects” ,Enter: Project-2,Save

Step 8: Test

1. Build **Project-1**,Jenkins will first run Project-1
2. After completion, **Project-2 will be triggered automatically**



Step 9: Configure Swap Memory(before jenkins install)

sudo fallocate -l 2G /swapfile, sudo chmod 600 /swapfile, sudo mkswap /swapfile
 sudo swapon /swapfile, echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab

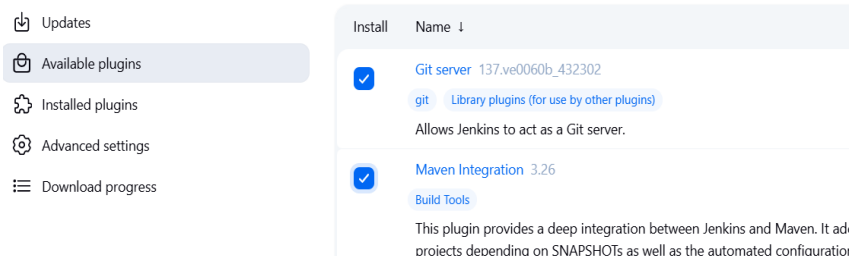
Task 5:Configure pollscm on the maven job and test whether a new build triggers if there is any change in the code.

Step 1: Install Required Jenkins Plugins

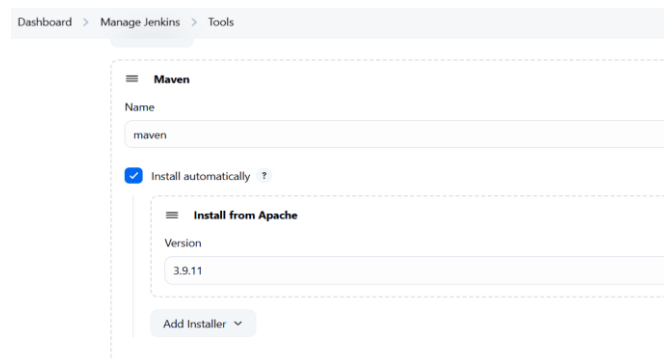
1. Go to Manage Jenkins > Manage PluginsUnder Available (or Installed tab):

- Git plugin
- Maven Integration plugin

2. Restart Jenkins if needed.



Step 2: Configure Maven (if not yet)



Step 3: Create the Maven Freestyle Job (Maven-PollSCM-Job)

Step 4: Configure Source Code Management

Dashboard > **Maven-PollSCM-Job** > Configuration

Configure

- General
- Source Code Management**
- Triggers
- Environment
- Build Steps
- Post-build Actions

☐ None
☒ **Git** ?

Repositories ?

Repository URL ?

https://github.com/avi913/Nokia-Project.git

Credentials ?

- none -

+ Add

Step 5: Configure Build Triggers (Poll SCM), Scroll to Build Triggers

1. Check "Poll SCM"

2. In the schedule box, enter: * * * * *

General

Source Code Management

Triggers

Environment

Build Steps

Post-build Actions

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

☐ Trigger builds remotely (e.g., from scripts) ?
☐ Build after other projects are built ?
☐ Build periodically ?
☐ GitHub hook trigger for GITScm polling ?
☒ **Poll SCM** ?

Schedule ?

* * * * *

Step 6: Test Auto-Build with Git Change: I have changed in line 5

avi913 / Nokia-Project

Issues Pull requests Actions Projects Wiki Security Insights

index.txt

ADME.md

index.txt

Nokia-Project / index.txt

avi913 Update index.txt

Code Blame 5 lines (5 loc) · 81 Bytes Code 55% faster

```

1 hello from feature branch
2 add the file
3 good morning
4 development
5 poll scm testing
  
```

Step 7: Configure the Build Save & Build

Dashboard > **Maven-PollSCM-Job** > #1 > Console Output

Status

Changes

Console Output

Edit Build Information

Delete build '#1'

Polling Log

Timings

Git Build Data

Console Output

Download

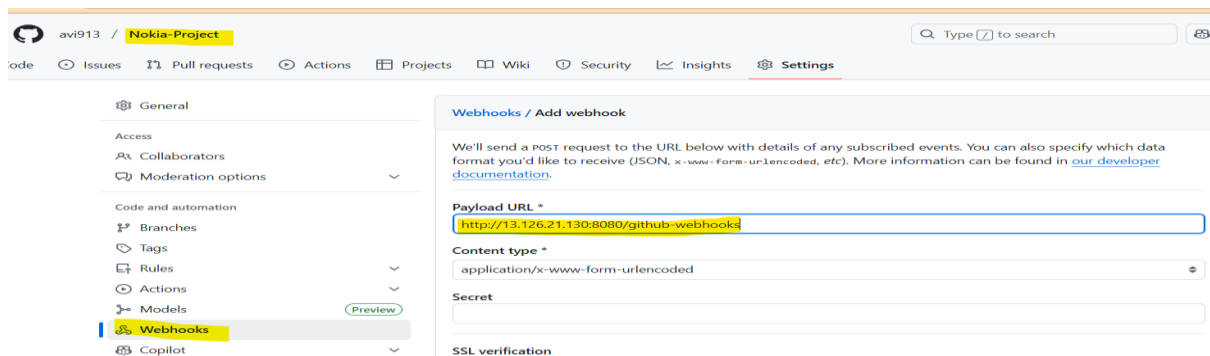
```

Started by an SCM change
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/Maven-PollSCM-Job
The recommended git tool is: NONE
No credentials specified
Cloning the remote git repository
Cloning repository https://github.com/avi913/Nokia-Project.git
> git init /var/lib/jenkins/workspace/Maven-PollSCM-Job # timeout=10
Fetching upstream changes from https://github.com/avi913/Nokia-Project.git
> git --version # timeout=10
> git --version # 'git version 2.43.0'
> git fetch --tags --force --progress -- https://github.com/avi913/Nokia-Project.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url https://github.com/avi913/Nokia-Project.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 2c5d90b100aa59d25da2fa8194ba80a7ac564a3 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 2c5d90b100aa59d25da2fa8194ba80a7ac564a3 # timeout=10
Commit message: "dev:03 modified file from main"
First time build. Skipping changelog.
Finished: SUCCESS
  
```

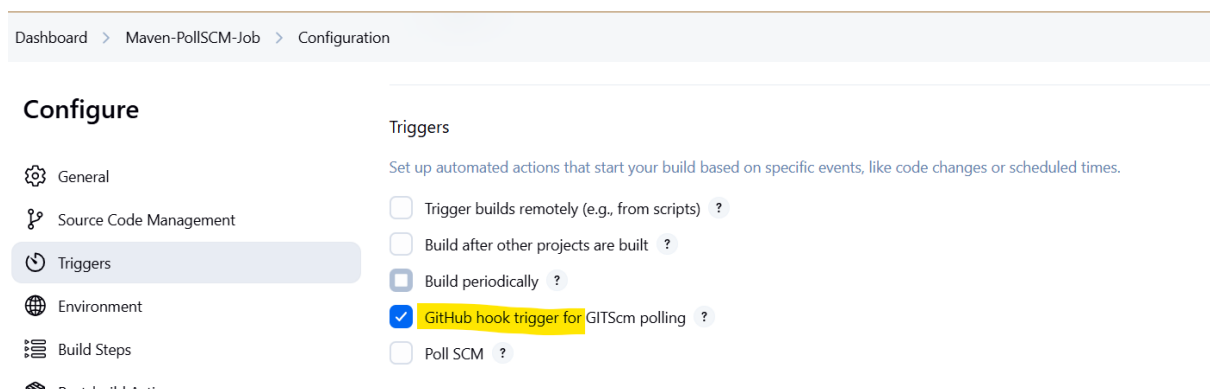
Task 6: Generate webhooks on github to trigger jenkins job whenever there is a push event on the repository,

Step 1: Go to the git repository – select setting – left side we have option called webhook – create new webhook

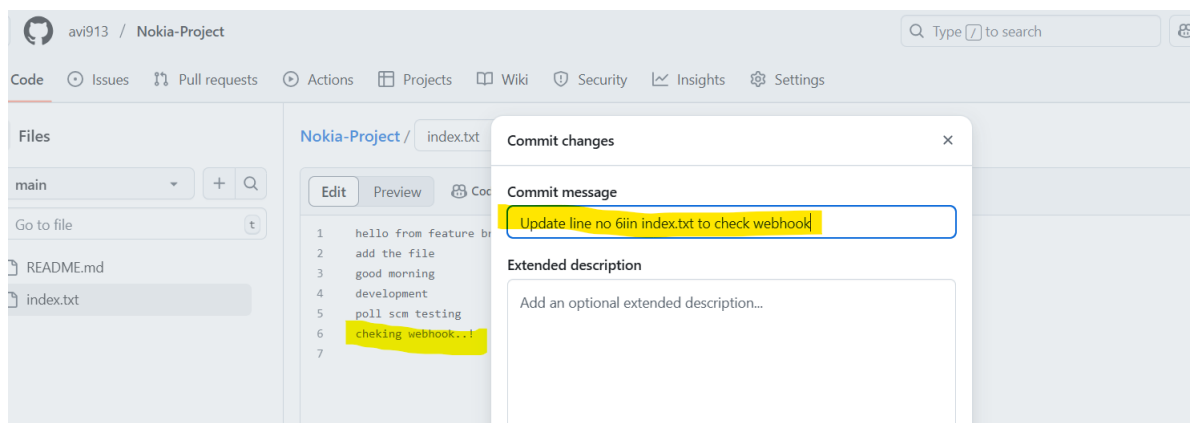
- There we need to add the webhook
- url : `http://13.126.21.130:8080/github-webhook/`



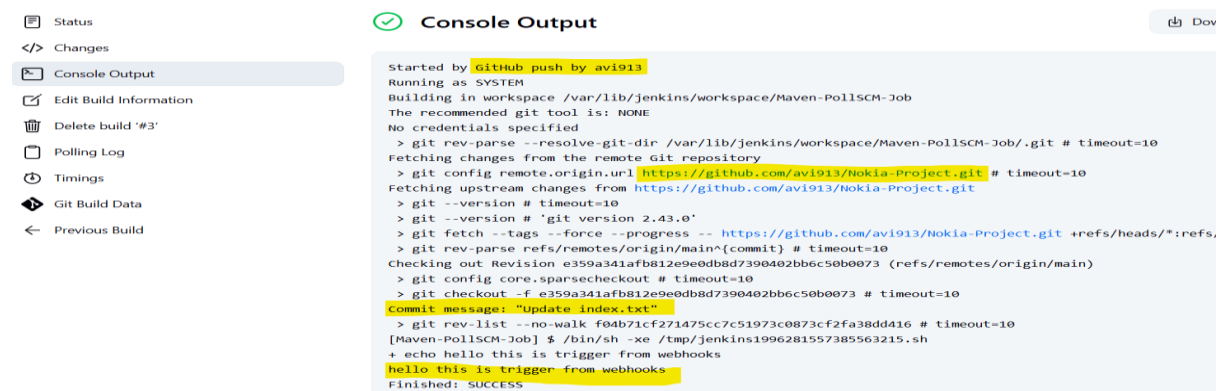
Step 2: Need to select githubhook to trigger-click save



Step 3: update the content in git hub repo -push the commits -webhook automatically trigger



See the webhook is trigger build no 3:



```
Status
</> Changes
Console Output
Edit Build Information
Delete build '#3'
Polling Log
Timings
Git Build Data
Previous Build

Console Output
Started by Github push by avi913
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/Maven-PollSCM-Job
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/Maven-PollSCM-Job/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/avi913/Nokia-Project.git # timeout=10
Fetching upstream changes from https://github.com/avi913/Nokia-Project.git
> git --version # timeout=10
> git --version # 'git version 2.43.0'
> git fetch --tags --force --progress -- https://github.com/avi913/Nokia-Project.git +refs/heads/*:refs,
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision e359a341afb812e9e0db8d7390402bb6c50b0073 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f e359a341afb812e9e0db8d7390402bb6c50b0073 # timeout=10
Commit message: "Update index.txt"
> git rev-list --no-walk f04b71cf271475cc7c51973c0873cf2fa38dd416 # timeout=10
[Maven-PollSCM-Job] $ /bin/sh -xe /tmp/jenkins1996281557385563215.sh
+ echo hello this is trigger from webhooks
hello this is trigger from webhooks
Finished: SUCCESS
```

Task 7:Examine pollscm and webhook and explain which is better and on what use cases you have to decide which one to use.

Poll SCM vs Webhook in Jenkins :

Feature	Poll SCM	Webhook
Definition	Jenkins checks the repo at fixed intervals	Repo (e.g. GitHub) notifies Jenkins on changes
Trigger	Scheduled polling (e.g., every 5 mins)	Real-time push from GitHub/GitLab
Speed	Slower (not instant)	Instant build trigger
Load	Jenkins checks repo even if no changes	Only runs when change happens (efficient)
Setup	Easy, works without public URL	Requires public IP or ngrok + webhook setup
Reliability	Always works, even if GitHub is slow	Webhook may fail if Jenkins is not reachable

Task 8:Work on pipeline as a code and configure maven jobs with Jenkinsfile using the official documentation here: <https://www.jenkins.io/doc/tutorials/build-a-java-app-with-maven/>

Install Required Plugins in Jenkins

Go to:Manage Jenkins → Manage Plugins → Available

Search and install:Pipeline,Pipeline: GitHub,GitHub Branch Source,Maven Integration,Blue Ocean

Create a New Pipeline Job

1. Go to **Jenkins Dashboard**, Click **New Item**, Name it something like: pipeline-avi-project, Select **"Pipeline"** → Click **OK**

New Item

Enter an item name

pipeline_avi_proj

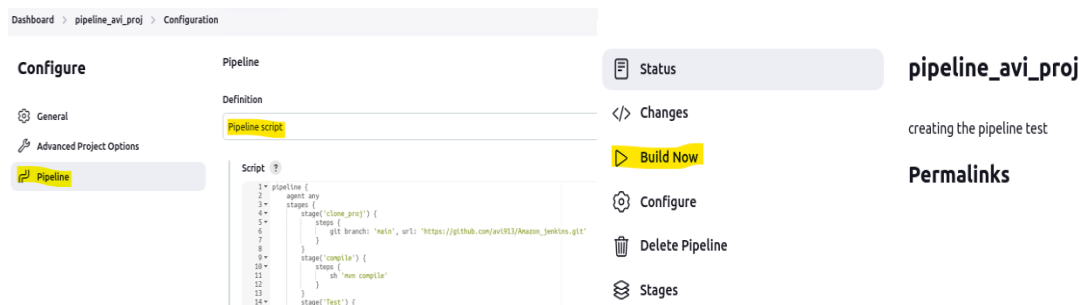
Select an item type

 **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

 **Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.

Configure the Pipeline: If Jenkinsfile is in the repo

- Under **Pipeline > Definition**: select Pipeline script from SCM
- **SCM: Git** Add your GitHub repo URL (e.g., https://github.com/avi913/Amazon_jenkins.git)



The screenshot shows the Jenkins configuration page for a new pipeline job named 'pipeline_avi_proj'. The 'Definition' section is set to 'Pipeline script'. The 'Script' field contains a Jenkinsfile script. The right sidebar shows the job's status as 'creating the pipeline test' and provides links for 'Status', 'Changes', 'Build Now', 'Configure', 'Delete Pipeline', and 'Stages'.

```
1 pipeline {
2   agent any
3   stages {
4     stage('clone_proj') {
5       steps {
6         git branch: 'main', url: 'https://github.com/avi913/Amazon_jenkins.git'
7       }
8     }
9     stage('compile') {
10      steps {
11        sh 'mvn compile'
12      }
13    }
14    stage('Test') {
15
```

pipeline {

agent any

stages {

stage('clone_proj') {

steps {

git branch: 'main', url: 'https://github.com/avi913/Amazon_jenkins.git'

}}

stage('compile') {

steps {

sh 'mvn compile'

}}

stage('Test') {

steps {

sh 'mvn test'

}}

```
stage('Deploy') {
    steps {
        sh 'mvn clean install'
    }
}
```

Task 9: Configure a multibranch project and test if new build happens post can with new branch creation

1. Create a Multibranch Pipeline Job

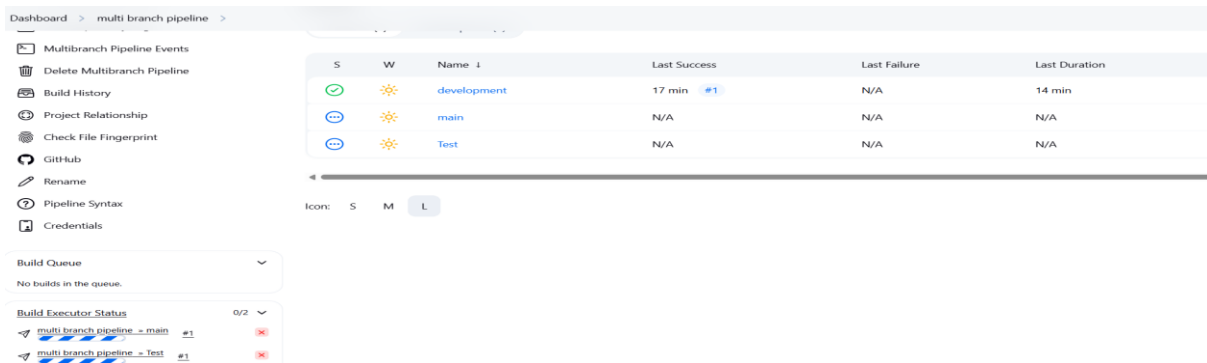
1. Go to **Dashboard** → **New Item**, Enter a name like My-MultiBranch-Project.
2. Select **Multibranch Pipeline**, Click **OK**.

2. Configure Source Repository

1. In the new job configuration: Go to **Branch Sources** → **Add Source** → **Git**.
2. Set the **Repository URL** (e.g., <https://github.com/user/repo.git>).
3. If needed, provide **credentials** (for private repos).
4. Under "**Discover Branches**", leave default Include all branches.

3. Scan Repository

- Click "**Scan Multibranch Pipeline Now**" (left menu).
- Jenkins will:
 - Detect all branches that contain a **Jenkinsfile**.



S	W	Name	Last Success	Last Failure	Last Duration
✓	☀	development	17 min #1	N/A	14 min
✓	☀	main	N/A	N/A	N/A
✓	☀	Test	N/A	N/A	N/A

- Trigger a build for each.

Task 10: Configure reverse proxy for your jenkins server using NGINX and secure the server by installing SSL certificate using certbot from Letsencrypt.

Prerequisites

- Jenkins running on port 8080 (default)
- Domain name (<https://urjenkins.duckdns.org/>) pointing to your EC2 public IP
- Port 80 (HTTP) and 443 (HTTPS) allowed in the Security Group
- NGINX and Certbot packages available

Step 1: Install NGINX

sudo apt update

sudo apt install nginx -y

```
Jul 22 05:33:04 ip-10-0-1-111 jenkins[519]: 2023-07-22 05:33:04.103400000 [id=23] INFO Hudson
Jul 22 05:33:04 ip-10-0-1-111 systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server
ubuntu@ip-10-0-1-111:~$ sudo apt update
sudo apt install nginx -y
Hit:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Ign:4 https://pkg.jenkins.io/debian-stable binary/ InRelease
Hit:5 https://pkg.jenkins.io/debian-stable binary/ Release
Hit:6 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
86 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

i-030185b5fc0366004 (demo-dc2)

Step 2: Configure Reverse Proxy: Create a new NGINX config for Jenkins:

sudo nano /etc/nginx/sites-available/Jenkins

Paste this configuration (replace with your actual domain):

```
server {  
    listen 80;  
    server_name urjenkins.duckdns.org;  
  
    location / {  
        proxy_pass http://localhost:8080;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
    }  
}
```

```
ubuntu@ip-10-0-1-111:~$ sudo cat /etc/nginx/sites-available/jenkins  
server {  
    server_name urjenkins.duckdns.org;  
  
    location / {  
        proxy_pass http://localhost:8080;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
    }  
}
```

Enable the configuration:

sudo ln -s /etc/nginx/sites-available/jenkins /etc/nginx/sites-enabled/

check the nginx status: sudo systemctl status nginx

```
ubuntu@ip-10-0-1-111:~$ sudo systemctl status nginx  
sudo lsof -i :80  
● nginx.service - A high performance web server and a reverse proxy server  
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)  
   Active: active (running) since Tue 2025-07-22 05:35:19 UTC; 21min ago  
     Docs: man:nginx(8)  
    Main PID: 2807 (nginx)  
      Tasks: 2 (limit: 1124)  
    Memory: 1.9M (peak: 4.5M)  
       CPU: 49ms  
    CGroup: /system.slice/nginx.service  
            └─2807 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"  
              └─3681 "nginx: worker process"
```

After Editing: Run these commands: Test & reload NGINX:

sudo nginx -t

sudo systemctl reload nginx

```

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-10-0-1-111:~$ sudo nano /etc/nginx/sites-available/jenkins
ubuntu@ip-10-0-1-111:~$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
ubuntu@ip-10-0-1-111:~$ sudo systemctl reload nginx
ubuntu@ip-10-0-1-111:~$ sudo ln -s /etc/nginx/sites-available/jenkins /etc/nginx/sites-enabled

```

Allow ufw: sudo ufw allow 8443 and 80

```

Status: active

To Action From
--
80/tcp ALLOW Anywhere
443 ALLOW Anywhere
80 ALLOW Anywhere
8080 ALLOW Anywhere
8443 ALLOW Anywhere
80/tcp (v6) ALLOW Anywhere (v6)
443 (v6) ALLOW Anywhere (v6)
80 (v6) ALLOW Anywhere (v6)
8080 (v6) ALLOW Anywhere (v6)
8443 (v6) ALLOW Anywhere (v6)

```

Step 3: Install Certbot and SSL (Let's Encrypt):

sudo apt install certbot python3-certbot-nginx -y

Run Certbot to get SSL:

sudo certbot --nginx -d urjenkins.duckdns.org

```

ubuntu@ip-10-0-1-111:~$ sudo certbot --nginx -d urjenkins.duckdns.org
Saving debug log to /var/log/letsencrypt/letsencrypt.log
Requesting a certificate for urjenkins.duckdns.org

Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/urjenkins.duckdns.org/fullchain.pem
Key is saved at: /etc/letsencrypt/live/urjenkins.duckdns.org/privkey.pem
This certificate expires on 2025-10-20.
These files will be updated when the certificate renews.
Certbot has set up a scheduled task to automatically renew this certificate in the background.

Deploying certificate
Successfully deployed certificate for urjenkins.duckdns.org to /etc/nginx/sites-enabled/jenkins
Congratulations! You have successfully enabled HTTPS on https://urjenkins.duckdns.org

-----
If you like Certbot, please consider supporting our work by:
 * Donating to ISRG / Let's Encrypt: https://letsencrypt.org/donate
 * Donating to EFF: https://eff.org/donate-le
-----
ubuntu@ip-10-0-1-111:~$ curl https://urjenkins.duckdns.org
<html><head><meta http-equiv='refresh' content='1;url=/login?from=%2F'><script id='redirect' data-redirect-url='/login?from=
d><body style='background-color:white; color:white;'>
Authentication required
<!--
-->

```

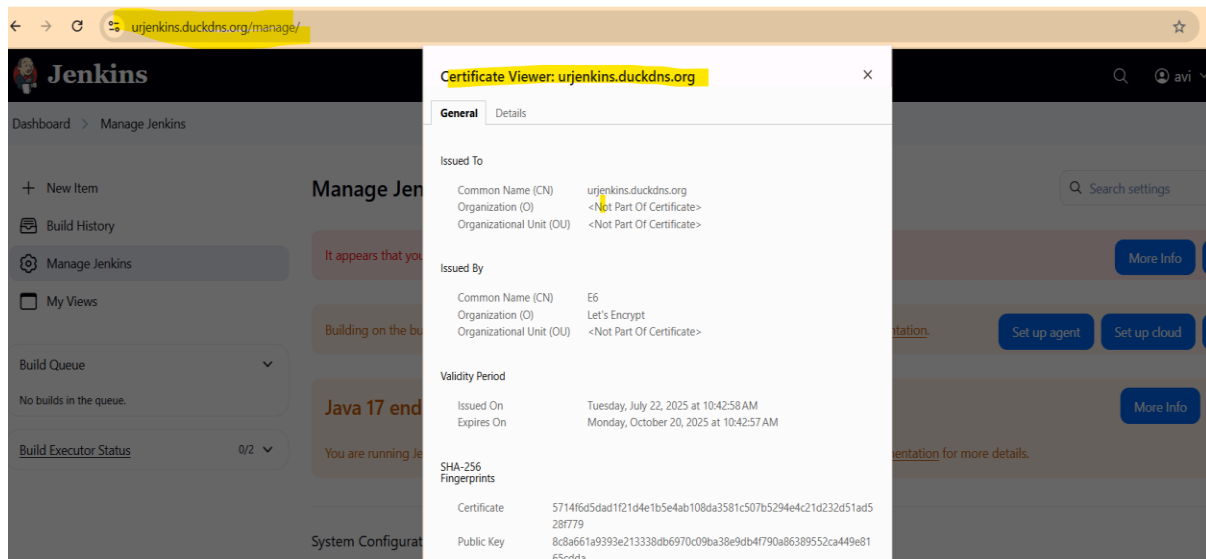
Follow the prompts:

- Agree to terms
- Provide your email
- Allow Certbot to update your NGINX config

After successful installation, SSL will be enabled

Step 4: Test HTTPS

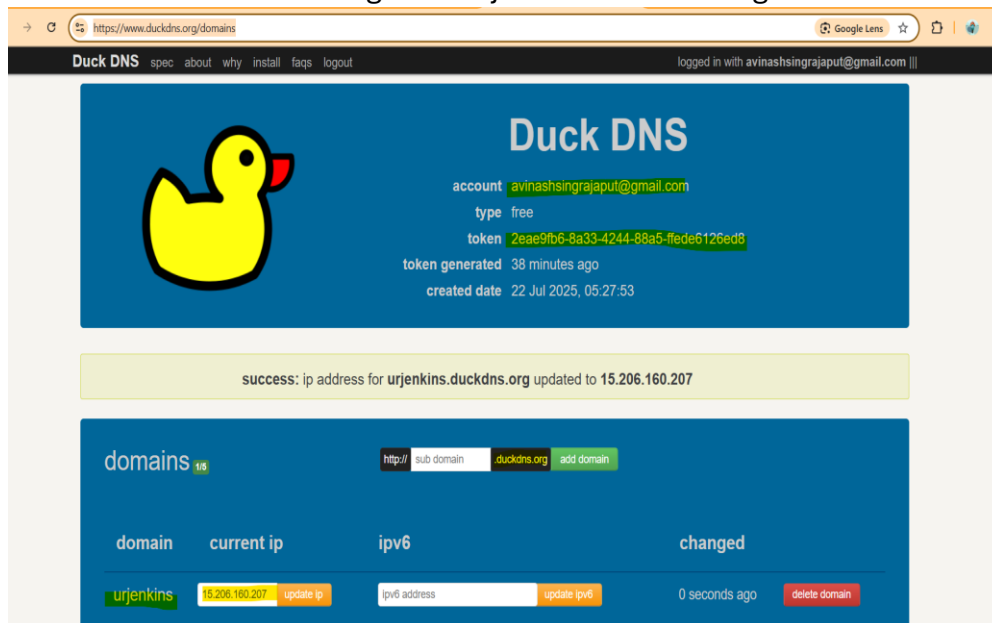
Go to: <https://urjenkins.duckdns.org/>



To generate free domain I have use duckdns :

Go to this link: <https://www.duckdns.org/>

- Sign in with GitHub/Google
- Choose a subdomain: e.g., avinashjenkins.duckdns.org
- Add your **EC2 public IP**
- Add this subdomain in your NGINX config
- Then run: `sudo certbot --nginx -d urjenkins.duckdns.org`



Note: Replace this duckDNS current ip with your EC2 public ip then only you can connect

Task 11: Create a Dockerfile for a simple application of your choice and run.

1. Simple Flask App for Demo

```
mkdir flask-app && cd flask-app
```

2. Create app.py

```
from flask import Flask
app = Flask(__name__)
@app.route('/')
def hello():
    return "Hello, Docker!"
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
~
~
~
```

3. Create requirements.txt

```
app.py requirements.txt
ubuntu@ip-10-0-1-89:~/flask-app$ cat requirements.txt
flask
ubuntu@ip-10-0-1-89:~/flask-app$
```

4. Create the Docker file

```
FROM python:3.9-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 5000
CMD ["python", "app.py"]
~
~
~
~
~
```

5. Build the image : Docker build -t flask-app-demo .

```

ubuntu@ip-10-0-1-89:~/flask-app$ docker build -t flask-app-demo
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  4.096kB
Step 1/6 : FROM python:3.9-slim
--> a1ec6a2ef164
Step 2/6 : WORKDIR /app
--> Using cache
--> 6c9c1962b452
Step 3/6 : COPY . .
--> Using cache
--> 048a9a7995c8
Step 4/6 : RUN pip install -r requirements.txt
--> Using cache
--> a1495243be65
Step 5/6 : EXPOSE 5000
--> Using cache
--> 7c98bd2843ab
Step 6/6 : CMD ["python", "app.py"]
--> Using cache
--> 45af91480089
Successfully built 45af91480089
Successfully tagged flask-app-demo:latest
ubuntu@ip-10-0-1-89:~/flask-app$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
ubuntu@ip-10-0-1-89:~/flask-app$ docker images
REPOSITORY    TAG          IMAGE ID            CREATED        SIZE
flask-app-demo latest       45af91480089        2 minutes ago 137MB
flask-app     1.0         45af91480089        2 minutes ago 137MB
python        3.9-slim    a1ec6a2ef164        6 weeks ago   126MB
ubuntu@ip-10-0-1-89:~/flask-app$

```

Run the image: `docker run -d -p 5000:5000 flask-app-demo`

Test the container: `curl http://localhost:5000`

```

See 'docker run --help'.
ubuntu@ip-10-0-1-89:~/flask-app$ docker run -d -p 5000:5000 flask-app-demo
f5e58062ad901a6b3b045a0f5b659bc4f529b86dcd38fd6cd6bef0de5ed86301
ubuntu@ip-10-0-1-89:~/flask-app$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
f5e58062ad90  flask-app-demo "python app.py"         4 seconds ago Up 3 seconds   0.0.0.0:5000->5000/tcp, :::5000->5000/tcp relaxed_yalow
ubuntu@ip-10-0-1-89:~/flask-app$ curl http://localhost:5000
Hello, Docker!
ubuntu@ip-10-0-1-89:~/flask-app$

```

Task:12 Create a multi-stage dockerfile by following the official document. [Multi-stage builds | Docker Docs](#)

1. we need a to files: app.py and requirements.txt

2. Multi-Stage Dockerfile :

FROM python:3.9-slim AS builder

WORKDIR /app

COPY requirements.txt .

RUN pip install --user -r requirements.txt

FROM python:3.9-slim

WORKDIR /app

COPY --from=builder /root/.local /root/.local

COPY . .

ENV PATH=/root/.local/bin:\$PATH

EXPOSE 5000

CMD ["python", "app.py"]

```
FROM python:3.9-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --user -r requirements.txt

FROM python:3.9-slim
WORKDIR /app

COPY --from=builder /root/.local /root/.local
COPY . .

ENV PATH=/root/.local/bin:$PATH

EXPOSE 5000
CMD ["python", "app.py"]
```

2. Build and Run:

`docker build -t flask-multistage-demo .`

`docker run -d -p 5000:5000 flask-multistage-demo`

```
Successfully built c20a22b1d5d2
Successfully tagged flask-multistage-demo:latest
ubuntu@ip-10-0-1-111:~/project$ docker images
REPOSITORY          TAG          IMAGE ID      CREATED        SIZE
flask-multistage-demo latest       c20a22b1d5d2  7 seconds ago  130MB
<none>              <none>      d817f0d74025  7 seconds ago  137MB
python              3.9-slim    a1ec6a2ef164  6 weeks ago    126MB
ubuntu@ip-10-0-1-111:~/project$
```

3. Test : curl http://localhost:5000

```
ubuntu@ip-10-0-1-111:~/project$ docker run -d -p 5000:5000 flask-multistage-demo
d6a2369e708e003bb1db7da86378fc18eb10710f50fa8c030883bd7ac773dded
ubuntu@ip-10-0-1-111:~/project$ curl http://localhost:5000
Hello from multi-stage Dockerfile!ubuntu@ip-10-0-1-111:~/project$
ubuntu@ip-10-0-1-111:~/project$
ubuntu@ip-10-0-1-111:~/project$ S
```

Task 13: Use Docker compose to run a multi container application

- `app.py` (Flask CRUD Example):
- file structure: multi-container-flask-app/
 `app/`
 `app.py`
 `requirements.txt`
 `Dockerfile`
 `docker-compose.yml/`

```
from flask import Flask
import pymysql

app = Flask(__name__)

@app.route('/')
def index():
    return "Hello from Flask with MySQL!"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

- Add the content in `requirements.txt` :
 Flask

Pymysql

```
ubuntu@ip-10-0-1-111:~/project$ cat requirements.txt
flask
pymysql
ubuntu@ip-10-0-1-111:~/project$
```

docker-compose.yml

```
version: '3.8'
services:
  web:
    build: ./app
    ports:
      - "5000:5000"
    depends_on:
      - db
  db:
    image: mysql:5.7
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: testdb
      MYSQL_USER: testuser
      MYSQL_PASSWORD: testpass
    ports:
      - "3306:3306"
```

Dockerfile

```
FROM python:3.9-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 5000
CMD ["python", "app.py"]
```

Run the Multi-Container App: docker-compose up -d

```
ubuntu@ip-10-0-1-111:~/multi-container-app$ tree app
app
├── Dockerfile
├── app.py
└── requirements.txt

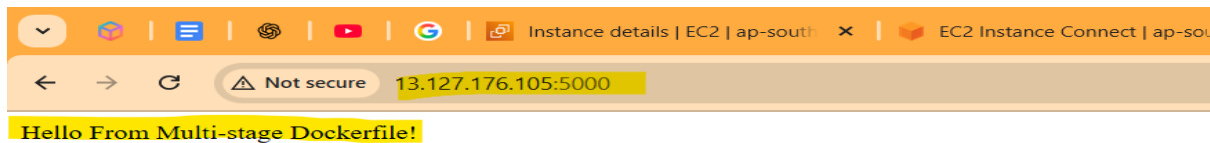
1 directory, 3 files
ubuntu@ip-10-0-1-111:~/multi-container-app$ ls
app  docker-compose.yml
ubuntu@ip-10-0-1-111:~/multi-container-app$ docker-compose up --build
Creating network "multi-container-app_default" with the default driver
Pulling db (mysql:5.7)...
5.7: Pulling from library/mysql
20e4dcae4c69: Pull complete
1c56c3d4ce74: Pull complete
e9f03a1c24ce: Pull complete
68c3898c2015: Pull complete
6b95a940e7b6: Pull complete
90986bb8de6e: Pull complete
ae71319cb779: Pull complete
ffc89e9dfd88: Pull complete
43d05e938198: Pull complete
064b2d298fba: Pull complete
258e1485562b: Pull complete
```

Test the connection in local: curl http://localhost:5000

```
ubuntu@ip-10-0-1-111:~/multi-container-app$ curl http://localhost:5000
Hello from multi-stage Dockerfile!ubuntu@ip-10-0-1-111:~/multi-container-app$
```

```
ubuntu@ip-10-0-1-111:~/multi-container-app$
ubuntu@ip-10-0-1-111:~/multi-container-app$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
5b72de221ed8   mysql:5.7     "docker-entrypoint.s..." 6 minutes ago  Up 6 minutes  0.0.0.0:3306->3306/tcp, :::3306->3306/tcp, 33060/tcp
d6a2369e708e   flask-multistage-demo "python app.py"          30 minutes ago  Up 30 minutes  0.0.0.0:5000->5000/tcp, :::5000->5000/tcp
ubuntu@ip-10-0-1-111:~/multi-container-app$
```

Run on Browser : http://13.127.176.105:5000



Task: 14 Install kubernetes cluster using kubeadm and also run some pods with yaml scripting

Plan Your EC2 Nodes:

Role	Size (min)	OS	Notes
Control plane (master)	t3.medium	Ubuntu 22.04 LTS	Needs 2 vCPU+, 2GB+ RAM.
Worker(s)	t3.small+	Ubuntu 22.04 LTS	At least 2GB RAM recommended.

Security Group ports (ingress)

- TCP 22 from your IP (SSH)
- TCP 6443 from worker node SG (Kubernetes API server)
- TCP 10250 from worker node SG (kubelet API)
- TCP 10259 (kube-scheduler), 10257 (kube-controller-manager) optional internal
- For node networking: allow all node-to-node traffic within cluster SG (simplest for lab).
- Open required pod network plugin ports (Calico uses BGP 179 between nodes; also VXLAN if enabled).
- sha256:e28120767d575d99cc56993eea2c2627d390dff46ab747b847fa37480671d633

Instances (2/3) Info								
Find Instance by attribute or tag (case-sensitive)					All states			
	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IP
☐	demo-dc2	i-030185b5fc0366004	Stopped	t2.micro	–	View alarms +	ap-south-1a	–
☒	k8s-master	i-07a1446a62a4c30b7	Running	t2.medium	Initializing	View alarms +	ap-south-1a	–
☒	k8s-worker	i-0a4c2f08ef083c8ea	Running	t2.medium	Initializing	View alarms +	ap-south-1a	–

Execute on Both "Master" & "Worker" Nodes

1. **Disable Swap: Required for Kubernetes to function correctly.**
`sudo swapoff -a`

2. Load Necessary Kernel Modules: Required for Kubernetes networking.

```
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
sudo modprobe overlay
sudo modprobe br_netfilter
```

```
ubuntu@ip-10-0-1-199:~$
ubuntu@ip-10-0-1-199:~$
ubuntu@ip-10-0-1-199:~$ sudo swapoff -a
ubuntu@ip-10-0-1-199:~$ cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
sudo modprobe overlay
sudo modprobe br_netfilter
overlay
br_netfilter
ubuntu@ip-10-0-1-199:~$ cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF
sudo sysctl --system
lsmod | grep br_netfilter
```

```
localhost:8080
E0721 10:26:39
localhost:8080
E0721 10:26:39
localhost:8080
E0721 10:26:39
localhost:8080
The connection
ubuntu@ip-10-0
E0721 10:26:45
localhost:8080
E0721 10:26:45
localhost:8080
E0721 10:26:45
localhost:8080
E0721 10:26:45
localhost:8080
The connection
ubuntu@ip-10-0
```

3. Set Sysctl Parameters: Helps with networking.

```
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF
sudo sysctl --system
lsmod | grep br_netfilter
lsmod | grep overlay
```

```
sudo modprobe br_netfilter
overlay
br_netfilter
ubuntu@ip-10-0-1-199:~$ cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF
sudo sysctl --system
lsmod | grep br_netfilter
lsmod | grep overlay
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
* Applying /usr/lib/sysctl.d/10-apparmor.conf ...
* Applying /etc/sysctl.d/10-bufferbloat.conf ...
* Applying /etc/sysctl.d/10-console-messages.conf ...
```

```
[kubectel-star
[kubectel-star
This node has
* Certificate
* The Kubelet
Run 'kubectl c
ubuntu@ip-10-0
E0721 10:26:39
localhost:8080
E0721 10:26:39
localhost:8080
E0721 10:26:39
localhost:8080
E0721 10:26:39
localhost:8080
```

4. Install Containerd:

```
sudo apt-get update
```

```
sudo apt-get install -y ca-certificates curl
```

```
sudo install -m 0755 -d /etc/apt/keyrings
```

```
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
```

```
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

```
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu
```

```
$(. /etc/os-release && echo \"${VERSION_CODENAME}\") stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
sudo apt-get update
```

```
sudo apt-get install -y containerd.io
```

```
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
br_netfilter
bridge                32768    0
overlaid               421888   1 br_netfilter
overlaid              212992    0
ubuntu@ip-10-0-1-199:~$ sudo apt-get update
sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.d
ocker.com/linux/ubuntu $(. /etc/os-release && echo \"${VERSION_CODENAME}\") stable" | sudo tee /etc/apt/s
ources.list.d/docker.list > /dev/null

sudo apt-get update
sudo apt-get install -y containerd.io

containerd config default | sed -e 's/SystemdCgroup = false/SystemdCgroup = true/' -e 's/sandbox_image
= "registry.k8s.io/pause:3.6"/sandbox_image = "registry.k8s.io/pause:3.9"/' | sudo tee /etc/container
d/config.toml

sudo systemctl restart containerd
sudo systemctl status containerd
Hit:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
```

5. Install Kubernetes Components:

```
sudo apt-get update
```

```
sudo apt-get install -y apt-transport-https ca-certificates curl gpg
```

```
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | sudo gpg --
dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

```
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
https://pkgs.k8s.io/core:/stable:/v1.29/deb/ /' | sudo tee
/etc/apt/sources.list.d/kubernetes.list
```

```
sudo apt-get update
```

```
sudo apt-get install -y kubelet kubeadm kubectl
```

```
sudo apt-mark hold kubelet kubeadm kubectl
```

```
containerd config default | sed -e 's/SystemdCgroup = false/SystemdCgroup =
true/' -e 's/sandbox_image = "registry.k8s.io/pause:3.6"/sandbox_image =
"registry.k8s.io/pause:3.9"/' | sudo tee /etc/containerd/config.toml
```

```
sudo systemctl restart containerd
```

```
sudo systemctl status containerd
```

```
io.containerd.timeout.shutdown = 3s
"io.containerd.timeout.task.state" = "2s"

[tttrpc]
address = ""
gid = 0
uid = 0
● containerd.service - containerd container runtime
   Loaded: loaded (/usr/lib/systemd/system/containerd.service; enabled; preset: enabled)
   Active: active (running) since Mon 2025-07-21 10:07:54 UTC; 25ms ago
     Docs: https://containerd.io
   Process: 2394 ExecStartPre=/sbin/modprobe overlay (code=exited, status=0/SUCCESS)
    Main PID: 2397 (containerd)
       Tasks: 7
      Memory: 14.0M (peak: 14.0M)
         CPU: 81ms
        CGroup: /system.slice/containerd.service
                └─2397 /usr/bin/containerd

Jul 21 10:07:54 ip-10-0-1-199 containerd[2397]: time="2025-07-21T10:07:54.156966177Z" level=info msg=
Jul 21 10:07:54 ip-10-0-1-199 containerd[2397]: time="2025-07-21T10:07:54.157025753Z" level=info msg=
Jul 21 10:07:54 ip-10-0-1-199 containerd[2397]: time="2025-07-21T10:07:54.157091282Z" level=info msg=

ubuntu@ip-10-0-1-199:~$ sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl gpg
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | sudo gpg --dearmor -o /etc/apt/key
rings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1
.29/deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list
sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
Hit:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 https://download.docker.com/linux/ubuntu noble InRelease
Hit:5 https://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
```

6. Execute ONLY on the "Master" Node

Initialize the Cluster: sudo kubeadm init

Set Up Local kubeconfig:

mkdir -p "\$HOME"/.kube

sudo cp -i /etc/kubernetes/admin.conf "\$HOME"/.kube/config

sudo chown "\$(id -u)": "\$(id -g)" "\$HOME"/.kube/config

Install a Network Plugin (Calico): kubectl apply -f

<https://raw.githubusercontent.com/projectcalico/calico/v3.26.0/manifests/calico.yaml>

Generate Join Command:

kubeadm token create --print-join-command

```
ubuntu@ip-10-0-1-199:~$ AC
ubuntu@ip-10-0-1-199:~$ sudo kubeadm token create --print-join-command
kubeadm join 10.0.1.199:6443 --token j50vv9.o0zz5lmxeny1xub1 --discovery-token-ca-cert-hash sha256:e28120767d575d99cc56993eea2c2627d390dff46ab747b847fa37480671d633
ubuntu@ip-10-0-1-199:~$ kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
ip-10-0-1-168       Ready     <none>    54s   v1.29.15
ip-10-0-1-199       Ready     control-plane 17m   v1.29.15
ubuntu@ip-10-0-1-199:~$
```

7. Execute on ALL of your Worker Nodes

Perform pre-flight checks: sudo kubeadm reset pre-flight checks

Paste the join command you got from the master node and append --v=5 at the end:

```
sudo kubeadm join 10.0.1.199:6443 \
--token j50vv9.o0zz5lmxeny1xub1 \
--discovery-token-ca-cert-hash
sha256:e28120767d575d99cc56993eea2c2627d390dff46ab747b847fa37480671d633 \
--cri-socket=unix:///run/containerd/containerd.sock
```

```
["unix:///run/containerd/containerd.sock" --v=5
flag needs an argument: --cri-socket
To see the stack trace of this error execute with --v=5 or higher
-bash: unix:///run/containerd/containerd.sock: No such file or directory
ubuntu@ip-10-0-1-168:~$ sudo kubeadm join 10.0.1.199:6443 \
--token j50vv9.o0zz5lmxeny1xub1 \
--discovery-token-ca-cert-hash sha256:e28120767d575d99cc56993eea2c2627d390dff46ab747b847fa37480671d633 \
--cri-socket=unix:///run/containerd/containerd.sock
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap...

This node has joined the cluster:
* Certificate signing request was sent to apiservert and a response was received.
* The Kubelet was informed of the new secure connection details.
```

8. Verify Cluster Connection & list all the slave container list

```
kubeadm join 10.0.1.199:6443 --token j50vv9.o0zz5lmxeny1xub1 --discovery-token-ca-cert-hash sha256:e28120767d575d99cc56993eea2c2627d390dff46ab747b847fa37480671d633
ubuntu@ip-10-0-1-199:~$ kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
ip-10-0-1-168       Ready     <none>    54s   v1.29.15
ip-10-0-1-199       Ready     control-plane 17m   v1.29.15
ubuntu@ip-10-0-1-199:~$
```

```

Last login: Mon Jul 21 09:56:18 2025 from 103.215.237.178
ubuntu@ip-10-0-1-168:~$
ubuntu@ip-10-0-1-168:~$ sudo ctr -n k8s.io containers list
CONTAINER                                IMAGE
RUNTIME
189fef7d36f4a627bb8c18cd9ca55080d41bc86ed5a2044b960556e40bc508e1  docker.io/calico/node:v3.26.0
io.containerd.runc.v2
89c867324bab6537a4e883e01ade0ee04f363191d4c0316f87eb231edd46350a  docker.io/calico/node:v3.26.0
io.containerd.runc.v2
9b593cb2b8be15111087c1334826fd2f8800da61c86dc2523eeb527e60b8c626  docker.io/calico/cni:v3.26.0
io.containerd.runc.v2
b941e9e3af816cc3d6767a6c9da34c7bd814a5efd2ba7cf7dd4cf9f5c7e62216  docker.io/calico/cni:v3.26.0
io.containerd.runc.v2
d43b633ee5d4b6613c31009ce526f42ff9ae147d9dd16442369c6c64f4076dc1  registry.k8s.io/pause:3.8
io.containerd.runc.v2
d45ef6caefd70c7c09bb3239a7b099be68fdbeb9c4589343264ca96d78c798f6  registry.k8s.io/pause:3.8
io.containerd.runc.v2
f5e13644df26cc63666d8b50d1c682b4f7789960d9f1fc3576ee87df95226108  registry.k8s.io/kube-proxy:v1.29.15
io.containerd.runc.v2
ubuntu@ip-10-0-1-168:~$

```

9. Deploy Test Workloads (YAML) apiVersion: apps/v1

kind: Deployment

metadata:

name: nginx-demo

spec:

replicas: 2

selector:

matchLabels:

app: nginx-demo

template:

metadata:

labels:

app: nginx-demo

spec:

containers:

- name: nginx

image: nginx:stable

ports:

- containerPort: 80

apiVersion: v1

kind: Service

metadata:

name: nginx-demo-svc

spec:

selector:

app: nginx-demo

ports:

- port: 80

targetPort: 80

protocol: TCP

type: ClusterIP Simple NGINX Deployment + Service (ClusterIP)

```
ubuntu@ip-10-0-1-199: ~  
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: nginx-demo  
spec:  
  replicas: 2  
  selector:  
    matchLabels:  
      app: nginx-demo  
  template:  
    metadata:  
      labels:  
        app: nginx-demo  
    spec:  
      containers:  
        - name: nginx  
          image: nginx:stable  
          ports:  
            - containerPort: 80  
---  
apiVersion: v1  
kind: Service  
metadata:  
  name: nginx-demo-svc  
spec:  
  selector:  
    app: nginx-demo  
  ports:  
    - port: 80  
      targetPort: 80  
      protocol: TCP  
  type: ClusterIP
```

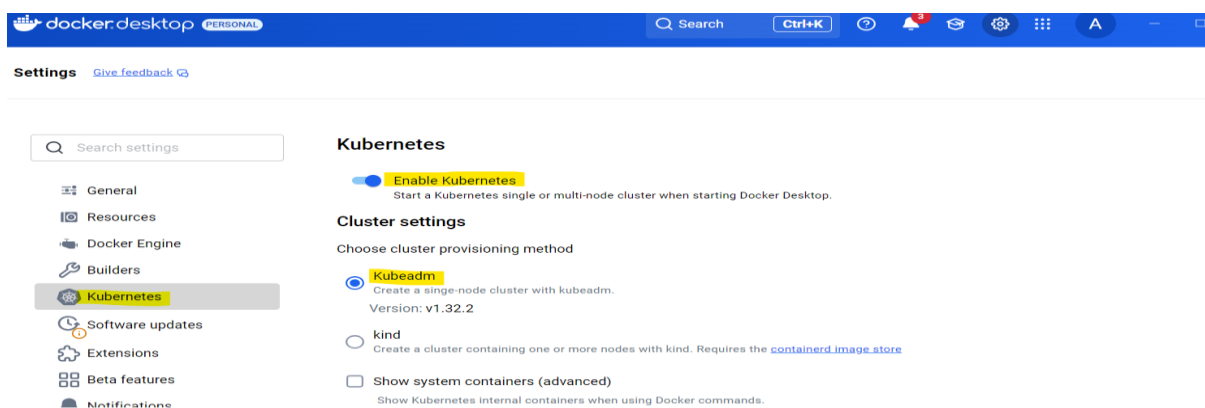
Then Apply: `kubectl apply -f nginx-deploy.yaml`

```
ubuntu@ip-10-0-1-199: ~  
ubuntu@ip-10-0-1-199:~$ ls  
ubuntu@ip-10-0-1-199:~$ vi nginx-deploy.yaml  
ubuntu@ip-10-0-1-199:~$ kubectl apply -f nginx-deploy.yaml  
deployment.apps/nginx-demo created  
service/nginx-demo-svc created  
ubuntu@ip-10-0-1-199:~$ kubectl get deploy,po,svc  
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE  
deployment.apps/nginx-demo          2/2      2              2             15s  
  
NAME                                READY    STATUS    RESTARTS    AGE  
pod/nginx-demo-5697d9f867-281xh     1/1      Running   0            15s  
pod/nginx-demo-5697d9f867-wjdgx     1/1      Running   0            15s  
  
NAME                                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE  
service/kubernetes                  ClusterIP    10.96.0.1     <none>         443/TCP    46m  
service/nginx-demo-svc             ClusterIP    10.104.64.58 <none>         80/TCP     15s  
ubuntu@ip-10-0-1-199:~$
```

Task:15 Install docker desktop and run a nginx deployment and expose using the service all steps are here:[k8s-learning/Readme.MD at main · mithuns1986/k8s-learning](https://github.com/mithuns1986/k8s-learning)

1. 📦 Install Docker Desktop (with Kubernetes enabled)

1. Download and install Docker Desktop from Docker's official website.
2. Open Docker Desktop → go to Settings → Kubernetes → Enable Kubernetes.
3. Apply changes → wait until Kubernetes is running (green status).



2. Prepare your Kubernetes manifest files

a) Create a Deployment → nginx-deployment.yaml

```
/usr/bin/bash --login -i
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
|
~
~
```

b) Create a Service → nginx-service.yaml

```
avinash@LAPTOP-420TVVSA ~/k8s $ cat nginx-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  type: NodePort
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080

avinash@LAPTOP-420TVVSA ~/k8s $ |
```

3. Apply the manifests to your cluster

kubectl apply -f nginx-deployment.yaml

kubectl apply -f nginx-service.yaml

```
avinash@LAPTOP-420TVVSA ~/k8s $ ls
nginx-deployment.yaml  nginx-service.yaml
avinash@LAPTOP-420TVVSA ~/k8s $ kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx-deployment created
avinash@LAPTOP-420TVVSA ~/k8s $ kubectl apply -f nginx-service.yaml
service/nginx-service created
avinash@LAPTOP-420TVVSA ~/k8s $
```

Verify deployment and service: Kubectl get all

```

service/nginx-service created
avinash@LAPTOP-420TVVSA ~/k8s $ kubectl get all
NAME                                READY    STATUS    RESTARTS   AGE
pod/nginx-deployment-647677fc66-hs47h 1/1      Running   0           97s
pod/nginx-deployment-647677fc66-tbn7m 1/1      Running   0           97s

NAME                                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
service/kubernetes                  ClusterIP     10.96.0.1     <none>         443/TCP          11m
service/nginx-service               NodePort      10.101.135.48 <none>         80:30080/TCP     85s

NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
deployment.apps/nginx-deployment 2/2      2              2            97s

NAME                                DESIRED    CURRENT    READY    AGE
replicaset.apps/nginx-deployment-647677fc66 2          2          2        97s

```

4. Access the NGINX Application

Since the Service is of type NodePort, you can access it via:



Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

Task:16 Create an voting app <https://github.com/mithuns1986/Deploy-VotingApp-on-Kubernetes.git>

1. Create Namespace

```
kubectl create namespace voting-app
```

```
kubectl config set-context --current --namespace=voting-app
```

2. Define Deployments for Each Component

Create the following *.yaml files. Replace image tags if needed.

```

avinash@LAPTOP-420TVVSA ~/project $ ls -lrt
total 13
drwxr-xr-x 1 vivek 197610  0 Aug  6 11:19 k8s-learning/
-rw-r--r-- 1 vivek 197610 361 Aug  6 11:22 '$'\302\226'
-rw-r--r-- 1 vivek 197610 200 Aug  6 11:23 redis-service.yaml
-rw-r--r-- 1 vivek 197610 454 Aug  6 11:23 voting-deployment.yaml
-rw-r--r-- 1 vivek 197610 218 Aug  6 11:24 voting-service.yaml
-rw-r--r-- 1 vivek 197610 462 Aug  6 11:24 worker-deployment.yaml
-rw-r--r-- 1 vivek 197610 538 Aug  6 11:27 db-deployment.yaml
-rw-r--r-- 1 vivek 197610 191 Aug  6 11:27 db-service.yaml
-rw-r--r-- 1 vivek 197610 450 Aug  6 11:28 result-deployment.yaml
-rw-r--r-- 1 vivek 197610 218 Aug  6 11:28 result-service.yaml

```

Applying Manifests :`kubectl apply -n vote -f .`

```

avinash@LAPTOP-420TVVSA ~/project $ vi namespace.yaml
avinash@LAPTOP-420TVVSA ~/project $ kubectl apply -f namespace.yaml
namespace/vote created
avinash@LAPTOP-420TVVSA ~/project $ kubectl get ns
NAME                STATUS    AGE
default             Active   47m
kube-node-lease     Active   47m
kube-public         Active   47m
kube-system         Active   47m
vote               Active   9s
voting-app         Active   18m
avinash@LAPTOP-420TVVSA ~/project $ kubectl apply -n vote -f all
error: the path "all" does not exist
avinash@LAPTOP-420TVVSA ~/project $ kubectl apply -n vote -f .
deployment.apps/db-deployment created
service/db-service created
namespace/vote unchanged
service/redis-service created
deployment.apps/result-deployment created
service/result-service created
deployment.apps/voting-deployment created
service/voting-service created
deployment.apps/worker-deployment created
avinash@LAPTOP-420TVVSA ~/project $ |

```

Verify Your Deployment: kubectl get deployments,pods,svc

```

avinash@LAPTOP-420TVVSA ~/proj/Deploy-VotingApp-on-Kubernetes/Deployment_Approach $ kubectl get all
NAME                                READY    STATUS    RESTARTS   AGE
pod/postgres-deploy-8c5cdd85c-s97kk 1/1      Running   0           2m28s
pod/redis-deploy-7456dd8d89-7gv28    1/1      Running   0           2m28s
pod/result-app-deploy-7d5bc77ff5-r5xm6 1/1      Running   0           2m28s
pod/voting-app-deploy-77dd96f74d-242kg 1/1      Running   0           2m27s
pod/worker-app-deployment-694d98b9d8-t55wh 1/1      Running   0           2m27s
pod/worker-app-deployment-694d98b9d8-xfh9f 1/1      Running   0           2m27s
pod/worker-app-deployment-694d98b9d8-zklvw 1/1      Running   0           2m27s

NAME                                TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
service/db                         ClusterIP    10.109.154.157   <none>            5432/TCP         2m28s
service/redis                     ClusterIP    10.98.221.192   <none>            6379/TCP         2m28s
service/result-service            NodePort     10.111.71.90    <none>            80:30005/TCP     2m28s
service/voting-service            NodePort     10.108.10.88    <none>            80:30004/TCP     2m27s

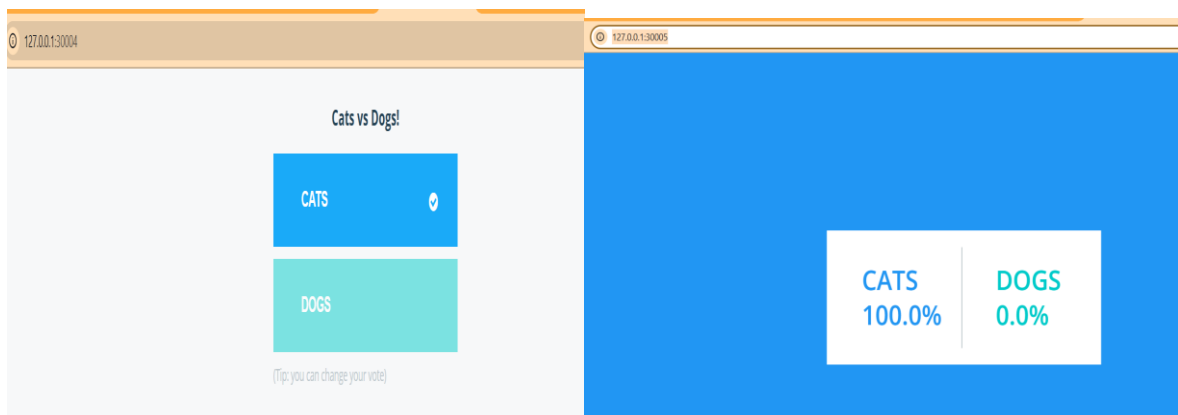
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
deployment.apps/postgres-deploy     1/1      1              1             2m28s
deployment.apps/redis-deploy        1/1      1              1             2m28s
deployment.apps/result-app-deploy    1/1      1              1             2m28s
deployment.apps/voting-app-deploy    1/1      1              1             2m28s
deployment.apps/worker-app-deployment 3/3      3              3             2m27s

NAME                                DESIRED    CURRENT    READY    AGE
replicaset.apps/postgres-deploy-8c5cdd85c 1          1          1        2m28s
replicaset.apps/redis-deploy-7456dd8d89    1          1          1        2m28s
replicaset.apps/result-app-deploy-7d5bc77ff5 1          1          1        2m28s
replicaset.apps/voting-app-deploy-77dd96f74d 1          1          1        2m27s
replicaset.apps/worker-app-deployment-694d98b9d8 3          3          3        2m27s

```

voting-service → NodePort: 30004

result-service → NodePort: 30005



Task 17: Create an EC2 using the terraform and migrate the statefile to Terraform Cloud of your organization in a new workspace.

Step 1: Configure AWS CLI(<https://awscli.amazonaws.com/AWSCLIV2.msi>)

Once installed, run: terraform configure

- AWS Access Key ID
- AWS Secret Access Key

- Default region name (us-east-1)
- Default output format (json)

```

history restored
PS C:\Users\vivek\Desktop\ec2-terraform> aws configure
AWS Access Key ID [None]: AKIAQWJF7NUOMT3FH6OU
AWS Secret Access Key [None]: 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
Default region name [None]: ap-south-1
Default output format [None]:
PS C:\Users\vivek\Desktop\ec2-terraform> aws ec2 describe-instances
{
  "Reservations": [

```

Step 2: Define Variables

```

EXPLORER
main.tf terraform.rc variables.tf
variables.tf > ...
1 variable "region" {
2   description = "AWS region"
3   type        = string
4   default     = "us-east-1"
5 }
6
7 variable "ami" {
8   description = "AMI ID"
9   type        = string
10 }
11
12 variable "instance_type" {
13   description = "Type of EC2 instance"
14   type        = string
15   default     = "t2.micro"
16 }
17
18 variable "instance_count" {
19   description = "Number of EC2 instances"
20   type        = number
21   default     = 1
22 }
23

```

Step 3: terraform.tfvars file

```

EXPLORER
main.tf terraform.rc variables.tf
terraform.tfvars > # instance_count
1 ami = "ami-0c1234567890abcde"
2 region = "us-east-1"
3 instance_type = "t2.micro"
4 instance_count = 1
5

```

Step 4: main.tf file

```

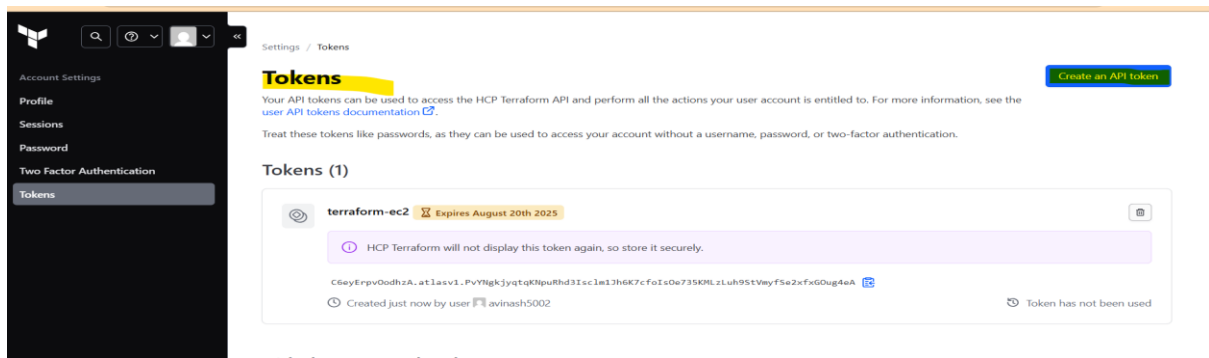
EXPLORER
main.tf terraform.rc variables.tf
main.tf > ...
1 terraform {
2   cloud {
3     organization = "Avinash-ABC"
4     workspaces {
5       name = "tf-workspace"
6     }
7   }
8 }
9
10 required_providers {
11   aws = {
12     source = "hashicorp/aws"
13     version = "~> 5.0" # or the version you want
14   }
15 }
16
17 provider "aws" {
18   region = var.region
19   # "example" (name)
20   # Reference Name
21 }
22 resource "aws_instance" "example" {
23   ami           = var.ami
24   instance_type = var.instance_type
25   count         = var.instance_count
26
27   tags = {
28     Name = "TerraformEC2"
29   }
30 }
31

```

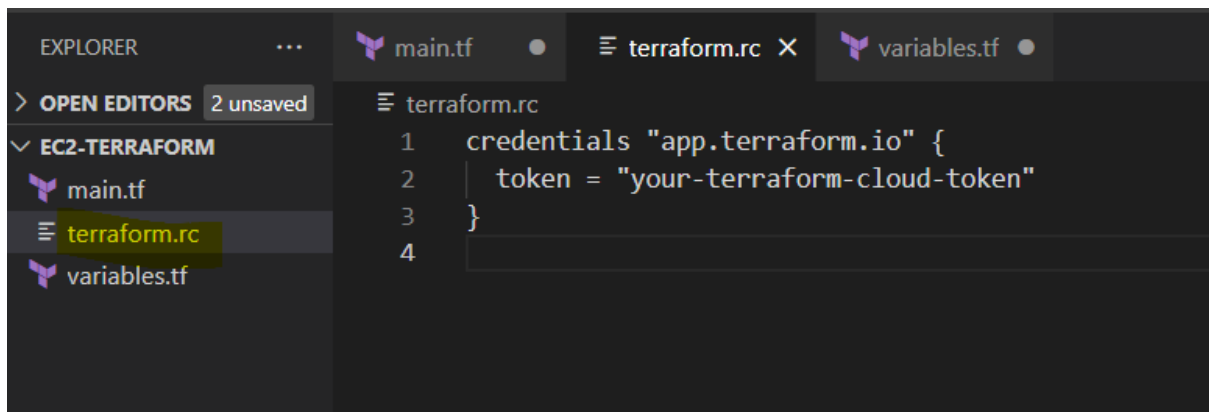
Step 5: Terraform Cloud Configuration

1. Go to Terraform Cloud
2. Create a new **organization** (if not already existing)
3. Create a **workspace** (e.g., ec2-project)
4. Get your **Terraform Cloud API Token**
Copy from: **User > Settings > Tokens**

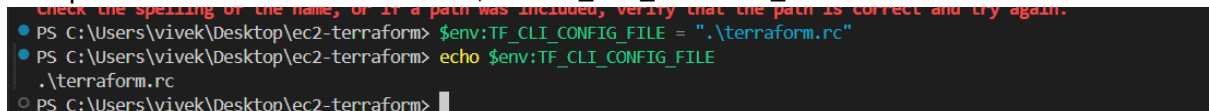
5. Create a file: terraform-ec2



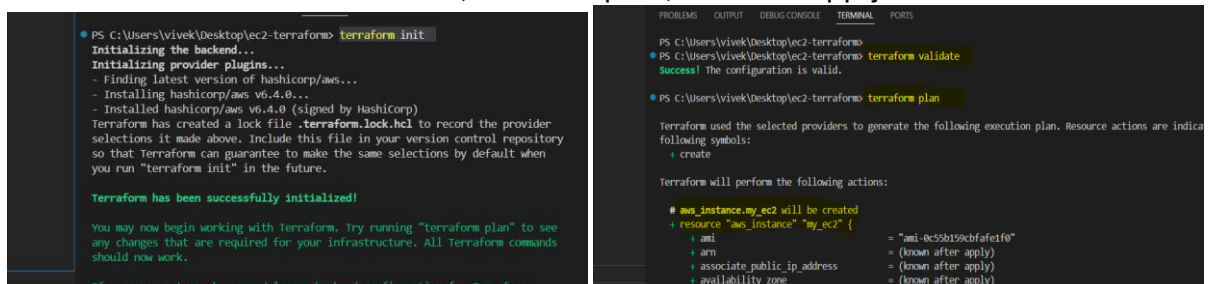
Create a file: terraform.rc



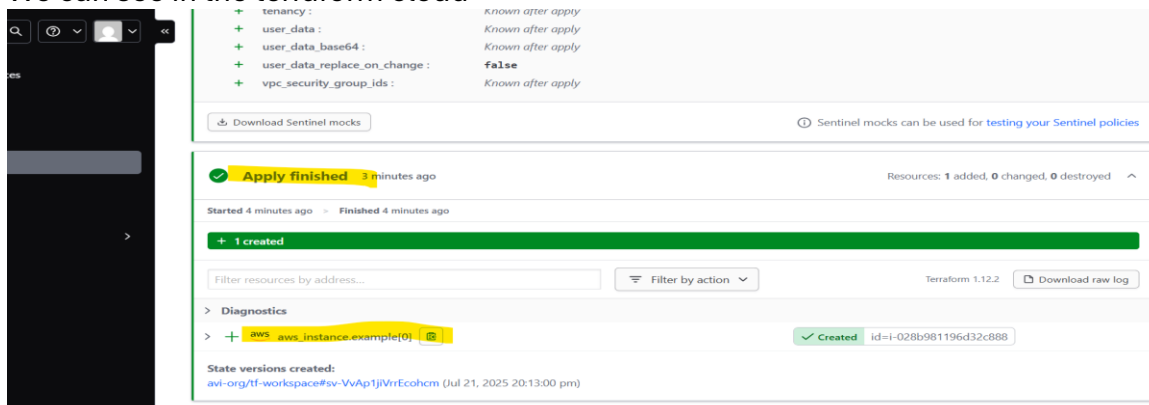
Run power shell cmd in terminal : `$env:TF_CLI_CONFIG_FILE = ".\terraform.rc"`



Commands to Run: terraform init, terraform plan, terraform apply



We can see in the terraform cloud



Ec2 is created in us-east-1 region

EC2 > Instances

Instances (1) Info

Find Instance by attribute or tag (case-sensitive) All states

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IP
TerraformEC2	i-028b981196d32c888	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1d	ec2-13-2

Terraform destroy

Workspace: terraform-destroy

Unlocked Resources: 0 Tags: 0 Terraform v1.12.2 Updated a few seconds ago

Current Run

Triggered via CLI CURRENT Applied a minute ago

Run List

All 6 Needs Attention 0 Errored 3 Running 0 On Hold 0 Success 3

Search Runs

Triggered via CLI CURRENT	destroy run	avinish5002 triggered via CLI	Applied 3 minutes ago
Triggered via CLI		avinish5002 triggered via CLI	Applied 5 minutes ago
Triggered via CLI	plan-only run	avinish5002 triggered via CLI	Planned and finished 7 minutes ago